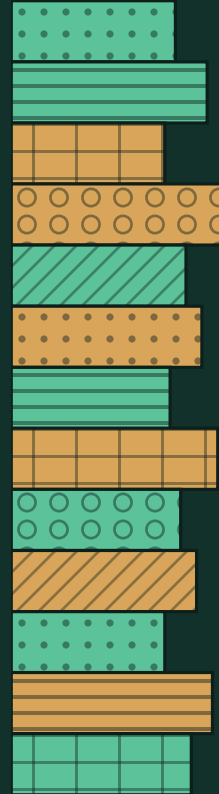


A FIELD GUIDE FOR SCIENTISTS WHO CAN USE A
COMPUTER BUT WANT TO UNDERSTAND ONE

The Missing Context

How computers actually work

*Written for Neha, a geologist learning the computer
from the bedrock up*



Shivam Meena

The Missing Context

How computers actually work

The Missing Context: How Computers Actually Work

Shivam Meena

First edition, September 2026. Written for Neha.

The book lives online at <https://claude.ai/artifact/SJnvHnQFpH9VWXKgPAG4zM> and may be updated after this PDF was made.

Companion site, with an assessment, a playground and a lab for every chapter:
<https://strata.theether.in>



Links in this PDF are clickable. Figures were drawn for this book. The sample data in Chapter 13 is invented for practice. Set in Lora, Bitstream Charter and DejaVu Sans Mono.

For Neha,

*who can read a landscape and deserves to read a computer the
same way.*

Contents

Preface: Why this book exists	6
Afterword: your own project	60
Appendix: What you should see	62
Cheat sheet: the commands you will keep looking up	64
Glossary	67
Further reading, in order	70
Books to grow into	71

Preface: Why this book exists

You already know how to learn hard things. A PhD in geology means you can read dense literature, hold a complex system in your head, and reason from evidence. None of that disappears when you sit in front of a computer. What is missing is not ability. It is **context**.

Most programming courses start with syntax: how to write a loop, how to define a function. They quietly assume you already know what a file path is, what the terminal does, why you "install" a library, and where your code actually runs. If you never picked up that background, every lesson feels like reading a paper in a field you have never studied. You can follow each sentence, but you cannot see why any of it matters.

This book fills in that background. It does not teach you a programming language. It teaches you the world that programming languages live in, so that when you do study a language, the pieces have somewhere to attach.

How to read this book

- **Read it in order.** Each chapter builds on the one before it, like stratigraphy: later layers make sense only once the earlier ones are laid down.
- **Do the "Try it" boxes.** Reading about the terminal teaches you far less than typing three commands into one. Each exercise takes a few minutes.
- **Don't memorise.** Aim to understand what each thing is *for*. Names and commands come with repetition, and you can always look them up (Chapter 10 shows you how).

- **Expect to re-read.** Some chapters will make much more sense after you have written a little code. That is normal.

Using the companion site

Every chapter ends with a link to [Neha's stratigraphic column](#), a companion site with three parts per chapter. Work them in this order:

1. **Read the chapter** here first, with the terminal closed. Just read.
2. **Assessment.** Five questions on the site; four right passes. Every answer, right or wrong, comes with an explanation, so the wrong ones teach more than the right ones.
3. **Playground.** Something to flip, type into or step through in the browser: switches and lamps, a practice terminal, Git's three trays. Nothing there can harm your computer, so experiment freely.
4. **On your computer.** The real thing on your own laptop. You type what you see and the site checks it.

Finish all three and the chapter's layer is deposited in your column. Budget about two hours for each of Chapters 1–4, three to four hours for each of Chapters 5–12, and an afternoon for the capstone. If you are stuck for more than twenty minutes, write down what you tried and move on; come back after the next chapter, when the missing piece has often arrived. Progress is saved in the browser you use, so stay with one browser, or use the site's *Copy progress* button to carry it to another.

A note on analogies

This book leans on comparisons to fieldwork and lab work because you already have a detailed mental model of those. Analogies are scaffolding, not truth. When one stops fitting, drop it and keep the idea underneath.

CHAPTER 1

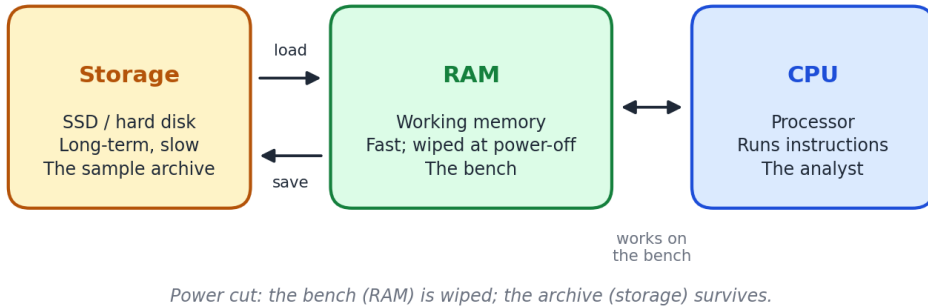
What a computer actually is

A computer is a machine that follows very simple instructions extremely fast. It does not understand anything. Everything it does, from playing a video to running a climate model, is built from a few basic operations: move a number, add numbers, compare numbers, and jump to a different instruction depending on the result.

The three parts that matter

Think of a small geochemistry lab with one analyst.

PART	ITS JOB	LAB ANALOGY
CPU	Runs instructions, billions per second	The analyst
RAM	Holds current work; wiped at power-off	The bench
Storage	Keeps everything long-term; slower	The archive

Figure 1. The three parts that matter**Figure 1.** Storage, RAM and CPU, with the archive, bench and analyst analogy

When you open a program, it is copied from storage onto the bench (RAM) so the analyst (CPU) can work with it quickly. When you "save" a file, you copy results from the bench back into the archive. This is why unsaved work is lost in a power cut: the bench is wiped, but the archive is not.

Everything is numbers

Inside the machine, everything is stored as **binary**: long sequences of 0s and 1s, called **bits**. Eight bits make a **byte**.

Text, images, sound and programs are all just agreed ways of turning information into numbers. The letter A is stored as 65. A pixel is three numbers for red, green and blue. A program is a sequence of numbers the CPU reads as instructions.

The meaning comes from **interpretation**, not from the bytes themselves. The same bytes could be read as text, as a picture or as garbage, depending on which program opens them. This is why opening a `.jpg` file in a text editor shows nonsense characters.

PLAYGROUND Flip eight switches on the companion site and watch the same lamps read as a number and as a letter: [Chapter 1 playground](#).

Sizes you will see

- **KB** (kilobyte) $\approx$ 1,000 bytes: a short text file
- **MB** (megabyte) $\approx$ 1 million bytes: a photo, a PDF
- **GB** (gigabyte) $\approx$ 1 billion bytes: RAM is usually 8–32 GB; a film is a few GB
- **TB** (terabyte) $\approx$ 1,000 GB: a large hard drive, or a big satellite imagery archive

■ Try it

Open your computer's system information (on Windows: *Settings* → *System* → *About*; on Mac: *Apple menu* → *About This Mac*). Find three numbers: your processor name, your RAM in GB, and your storage size. You now know the bench size and archive size of your own lab.

■ Watch alongside

Each link opens a YouTube search for the exact title; the first result is the video. The full series is the [Crash Course Computer Science playlist](#).

- [Crash Course CS #4: Representing Numbers and Letters with Binary](#) (11 min)
- [Crash Course CS #6: Registers and RAM](#) (12 min)
- [Crash Course CS #7: The Central Processing Unit](#) (12 min)
- [Crash Course CS #19: Memory and Storage](#) (12 min)

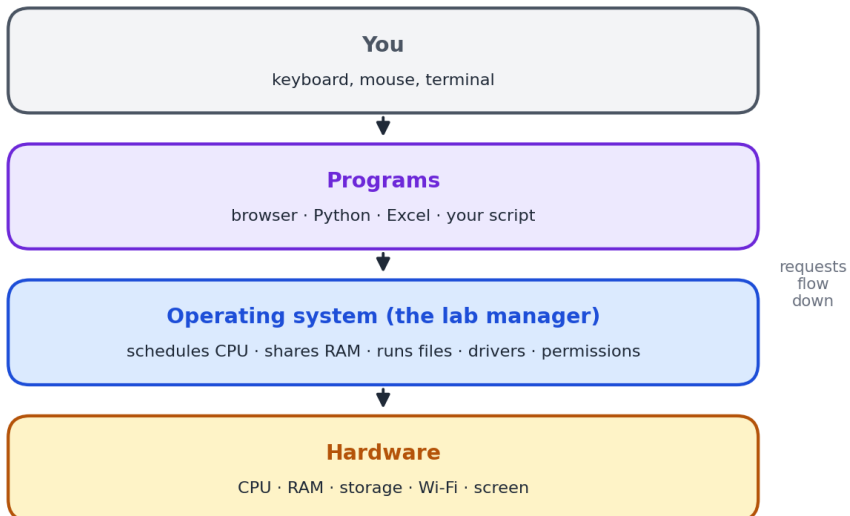
ON THE COMPANION SITE Chapter 1 on the companion site

CHAPTER 2

The operating system

The **operating system (OS)** is the program that runs the computer on behalf of every other program. Windows, macOS, Linux, Android and iOS are all operating systems. You never really use the hardware directly; you use the OS, and the OS uses the hardware.

Figure 2. Everything goes through the operating system



A program never touches hardware directly: it asks the OS (a "system call").

Figure 2. You, programs, the operating system and hardware as layers, with requests flowing down

The lab manager

If the CPU is the analyst, the operating system is the **lab manager**. Many projects (programs) want the analyst's time, bench space and instruments at once. The lab manager:

- **Schedules time.** It switches the CPU between programs thousands of times per second, so a browser, a music player and your code all appear to run together.
- **Allocates bench space.** It gives each program its own portion of RAM and stops programs from touching each other's space.
- **Runs the archive.** It organises storage into files and folders (Chapter 3) and controls who may read or change them.
- **Controls the instruments.** Keyboard, screen, Wi-Fi, printer and USB drives are all reached through the OS using small programs called **drivers**.
- **Checks permissions.** It knows which user is logged in and what they are allowed to do. This is why installing some software asks for an administrator password.

PLAYGROUND Watch one CPU share itself between three programs, then route requests to the lab manager's five jobs: [Chapter 2 playground](#).

What the OS gives to programs

This part matters most for programming. A program cannot open a file or connect to the internet on its own. It **asks the OS** to do it, through a fixed set of requests called **system calls**. When your Python code says `open("data.csv")`, Python passes that request to the OS, and the OS finds the file on disk.

This is also why a program built for Windows usually does not run on macOS. The two lab managers accept different request forms.

Programs and processes

- A **program** is the set of instructions sitting in storage, like a written lab protocol.
- A **process** is that program actually running, with its own memory, like the protocol being carried out today.

You can run the same program several times, giving several processes. Opening two browser windows often creates many processes.

The three families you will meet

OS	WHERE YOU SEE IT	PROGRAMMER'S NOTE
Windows	Most laptops in India	Paths use <code>\</code>
macOS	Apple computers	Unix-based, like Linux
Linux	Servers, clusters, Android	Most tutorials assume it

If you ever run models on a university cluster, it will almost certainly be Linux. That is why tutorials keep showing Linux commands.

Try it

Open **Task Manager** (Windows: `Ctrl + Shift + Esc`) or **Activity Monitor** (Mac: search in Spotlight). Look at the list of processes and the CPU and memory columns. Open a browser tab and watch a new process appear. You are looking directly at the lab manager's schedule.

Watch alongside

- Crash Course CS #18: Operating Systems (14 min)
- Crash Course CS #8: Instructions and Programs (11 min)

ON THE COMPANION SITE [Chapter 2 on the companion site](#)

CHAPTER 3

Files, folders and paths

A **file** is a named chunk of bytes in storage. A **folder** (also called a **directory**) is a container that holds files and other folders. Almost every beginner error of the "file not found" kind comes from not having a clear picture of this chapter.

The tree

Storage is organised as a **tree**, like a core-sample archive: building → room → shelf → box → sample. At the very top is the **root**. On Windows each drive has its own root, such as `C:\`. On macOS and Linux there is a single root, written `/`.

```
C:\ (root)
├── Users
│   └── Neha
│       ├── thesis
│       │   ├── chapter1.docx
│       │   └── data
│       │       └── boreholes.csv
│       └── mca
│           └── assignment1.py
```

Paths: the address of a file

A **path** is the route through the tree to one file, written with separators between the levels.

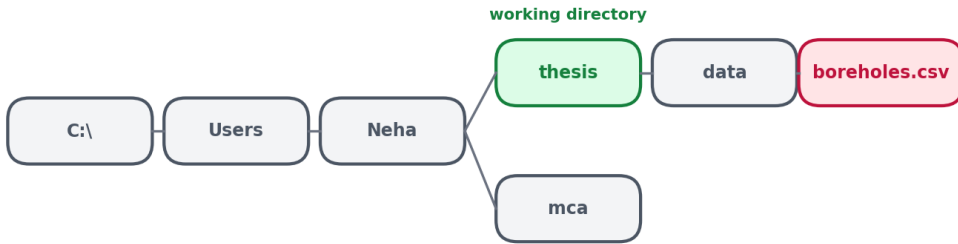
- Windows: `C:\Users\Neha\thesis\data\boreholes.csv`
- macOS / Linux: `/Users/neha/thesis/data/boreholes.csv`

Windows separates levels with a backslash `\`; macOS and Linux use a forward slash `/`. Python accepts forward slashes on Windows too, which is why many tutorials use them everywhere.

There are two kinds of path, and the difference causes endless confusion:

- An **absolute path** starts at the root. It works from anywhere, like a full grid reference.
- A **relative path** starts from *where you currently are*, called the **working directory**. It is like saying "two boxes to the left": correct only if you know where you are standing.

If your code is running in `C:\Users\Neha\thesis`, then the relative path `data\boreholes.csv` finds the file. Run the same code from `C:\Users\Neha\mca` and it fails, even though nothing about the file changed. When you see `FileNotFoundError`, the first question is always: **where is my program running from?**

Figure 3. Absolute and relative paths

Absolute (from the root, works from anywhere):

`C:\Users\Neha\thesis\data\boreholes.csv`

Relative (from the working directory, here thesis):

`data\boreholes.csv`

Run the same code from mca and this relative path fails: `FileNotFoundError`

Figure 3. Neha's folder tree showing the absolute path and the relative path from the thesis folder

PLAYGROUND Stand in a folder, type a path and follow it step by step until it lands, or breaks: [Chapter 3 playground](#).

Two special shortcuts appear everywhere:

- `.` means "the folder I am in"
- `..` means "the folder one level up"

File extensions

The ending of a name, like `.csv`, `.py` or `.pdf`, is the **extension**. It is only a label that tells the OS which program to open the file with. Renaming `data.csv` to `data.pdf` does not turn it into a PDF, any more than relabelling a jar changes the rock inside.

Some extensions you will meet constantly:

EXTENSION	WHAT IT IS
<code>.txt</code> , <code>.md</code>	Plain text, readable by any editor
<code>.csv</code>	A table stored as plain text, values separated by commas
<code>.py</code> , <code>.java</code> , <code>.c</code> , <code>.js</code>	Source code (plain text) in Python, Java, C, JavaScript
<code>.json</code>	Structured data as plain text, common for settings and web data
<code>.exe</code> (Windows), <code>.app</code> (Mac)	Programs ready to run
<code>.zip</code>	A compressed bundle of other files

Notice how many of these are **plain text**. Code is just text in a file. Word documents are not plain text, which is why you must never write code in Microsoft Word.

Hidden files

Files whose names start with a dot, such as `.gitignore` or `.env`, are hidden by default. Programming tools often store settings in them. If a tutorial mentions a file you cannot see, turn on "show hidden files" in your file explorer.

Try it

In your file explorer, open any document and find its full path. On Windows, click the address bar; on Mac, right-click the file while holding Option and choose *Copy as Pathname*. Paste it somewhere and read it level by level from the root down.

Watch alongside

- [Crash Course CS #20: Files and File Systems](#) (12 min)
- [Crash Course CS #21: Compression](#) (13 min), optional, explains what `.zip` and `.jpg` actually do

ON THE COMPANION SITE [Chapter 3 on the companion site](#)

CHAPTER 4

The terminal

The **terminal** (also called the command line, shell, console or command prompt) is a way of giving the operating system instructions by typing instead of clicking. It looks old-fashioned, but it is the main working tool of programmers, because it is precise, repeatable and scriptable.

Why not just click?

Clicking is like describing a field route by pointing at a map. Typing a command is like writing down the exact coordinates. A written command can be copied, shared, repeated a thousand times, and put in a tutorial. That is why every installation guide says "run this command" rather than "click here".

Reading a command

Every command has the same shape:

command	options	arguments
ls	-l	thesis/data

- **command**: the program to run (ls means "list")
- **options** (or flags): adjust how it behaves, usually starting with `-` or `--` (`-l` means "long format, with details")

- **arguments:** what to act on (the folder `thesis/data`)

The **prompt** is the text before your cursor, often ending in `$` or `>`. It usually shows your current working directory. In tutorials, `$ ls` means "type `ls`"; do not type the `$`.

The ten commands that cover most daily use

TASK	WINDOWS (POWERSHELL)	MACOS / LINUX
Where am I?	<code>pwd</code>	<code>pwd</code>
List files here	<code>ls</code> or <code>dir</code>	<code>ls</code>
Go into a folder	<code>cd thesis</code>	<code>cd thesis</code>
Go up one level	<code>cd ..</code>	<code>cd ..</code>
Make a folder	<code>mkdir results</code>	<code>mkdir results</code>
Show a text file	<code>cat notes.txt</code>	<code>cat notes.txt</code>
Copy a file	<code>cp a.csv b.csv</code>	<code>cp a.csv b.csv</code>
Move or rename	<code>mv a.csv data\</code>	<code>mv a.csv data/</code>
Delete a file	<code>rm old.csv</code>	<code>rm old.csv</code>
Run a Python file	<code>python script.py</code>	<code>python3 script.py</code>

PLAYGROUND A practice PowerShell with a pretend C: drive. Try every command in the table there before touching your real one: [Chapter 4 playground](#).

Two warnings. First, `rm` deletes permanently; there is no Recycle Bin. Second, the terminal is silent when things succeed. No message usually means it worked.

Time-savers

- Press **Tab** to auto-complete file and folder names. This also prevents typos.

- Press the **up arrow** to bring back previous commands.
- Press **Ctrl + C** to stop a command that is running or stuck.

What actually happens when you type `python script.py`

The shell looks for a program called `python` in a list of folders called the **PATH** (an OS setting, not the same as a file path). If it finds one, it starts it as a new process and passes it `script.py` as an argument. Python then reads your file and runs it.

The common error `'python' is not recognized or command not found` means exactly one thing: the shell searched the PATH and did not find that program. Either it is not installed, or its folder was never added to the PATH. You will meet this at least once, and now you will know what it means.

Try it

Open a terminal (Windows: search for *PowerShell*; Mac: search for *Terminal*). Type `pwd`, then `ls`, then `cd` into one of the folders you see, and `ls` again. Then `cd ..` to come back. Draw the part of the tree you just walked through.

Watch alongside

- [Crash Course CS #22: Keyboards and Command Line Interfaces](#) (11 min)
- [MIT Missing Semester, Lecture 1: Course Overview + The Shell](#) (about 50 min; notes on the [course site](#))
- Practise with the [Software Carpentry Unix Shell lesson](#), written for researchers

ON THE COMPANION SITE [Chapter 4 on the companion site](#)

CHAPTER 5

Setting up your computer

You have now met the operating system, paths and the terminal. Next, install three tools and check each one works: **Python** (the interpreter that runs code), **VS Code** (the editor you write code in) and **Jupyter** (notebooks, used widely in science). Most beginners get stuck here, not on the code itself, so take your time and check each step. Instructions are Windows first, with Mac notes.

Two words appear here before their own chapters. The *interpreter* is the program that reads and runs your code; you'll meet it properly in Chapter 6. *pip* is Python's package installer; you'll meet it in Chapter 7. For now, follow the steps like a lab protocol. Chapters 6–8 explain *why* each one works.

Step 1: Install Python

1. Go to python.org/downloads and download the latest Python 3 installer.
2. Run it. On the **first screen**, tick "**Add python.exe to PATH**" at the bottom before clicking *Install Now*. This single checkbox prevents the most common beginner error (Chapter 4 explains PATH).
3. When it finishes, close and reopen any open terminal windows.

Mac: download the macOS installer from the same page and run it. On a Mac, type `python3` and `pip3` wherever this book says `python` and `pip`.

Step 2: Check Python works

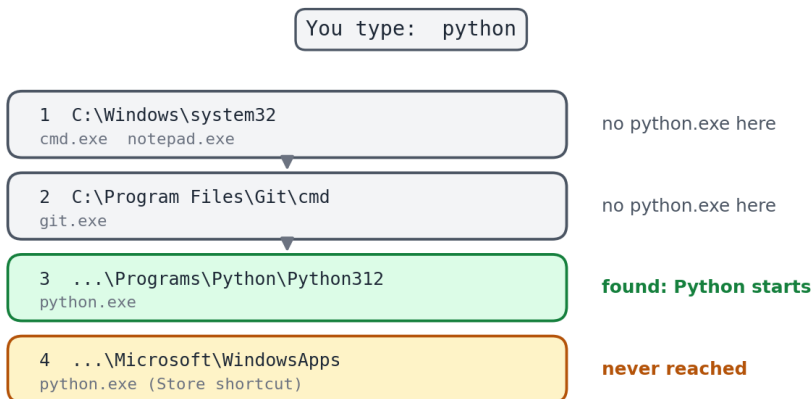
Open **PowerShell** (press the Windows key, type *PowerShell*, press Enter) and type:

```
python --version
pip --version
```

You should see something like `Python 3.12.6` and a `pip` line ending in `(python 3.12)`. The exact numbers do not matter; seeing a version does.

If typing `python` opens the **Microsoft Store** instead, Windows is using a shortcut. Turn it off in *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, switching off both "python.exe" and "python3.exe", then open a new PowerShell window.

How the shell finds a command: it checks each PATH folder in order



Untick "Add python.exe to PATH" and folder 3 is missing: the search reaches folder 4 and opens the Microsoft Store, or ends with "'python' is not recognized".

Figure 9. How the shell searches the PATH folders in order and stops at the first python.exe it finds

PLAYGROUND Tick and untick the installer's PATH box and watch the shell search folder by folder: [Chapter 5 playground](#).

Step 3: Install VS Code

1. Download it from code.visualstudio.com and install it with the default options.
2. Open VS Code, click the **Extensions** icon on the left (four small squares), and install **Python** and **Jupyter**, both published by Microsoft.

Step 4: Install the packages this book uses

In PowerShell, run:

```
python -m pip install ipykernel pandas matplotlib requests
```

`ipykernel` lets Jupyter notebooks run Python. The others are introduced in Chapters 7, 9 and 13. The download takes a few minutes.

Step 5: Make a home for your work

Create a folder called `learning` inside your *Documents* folder. In VS Code choose **File** → **Open Folder** and open it. Always open a *folder* in VS Code, not single files. This keeps the working directory predictable (Chapter 3).

Step 6: Run your first program

1. In VS Code, create a new file called `hello.py` inside `learning`.
2. Type this line and save:

```
print("Hello, Neha. Python is working.")
```

3. Open VS Code's built-in terminal with **Terminal** → **New Terminal** and run:

```
python hello.py
```

If the message appears, your setup is complete.

If something goes wrong

YOU SEE	FIX
<code>'python'</code> is not recognized	PATH was not set. Re-run the installer, choose <i>Modify</i> , and tick "Add to environment variables". Then reopen the terminal.
The Microsoft Store opens	Turn off the app execution aliases (Step 2).
<code>pip</code> works but packages are "missing"	Always use <code>python -m pip install ...</code> (Chapter 8 explains why).
VS Code asks to "select interpreter"	Click it and choose the Python version you installed.

Try it

Change the message in `hello.py` to include today's date and run it again. Then, in VS Code's terminal, run `python --version` and note the version number. You will use it in the Chapter 5 lab.

Watch alongside

- Corey Schafer: Python Tutorial for Beginners 1: Install and Setup for Mac and Windows (about 20 min)
- Corey Schafer: Visual Studio Code (Windows): Setting up a Python Development Environment (watch the first half)

ON THE COMPANION SITE [Chapter 5 on the companion site](#)

CHAPTER 6

What a programming language actually is

A programming language is a **precise written notation for instructions**, designed so that a human can write it and a program can translate it into the numbers a CPU understands. It is closer to a lab protocol written in strict technical shorthand than to a spoken language.

Why languages exist at all

The CPU only understands **machine code**: raw numbers meaning "load this", "add that", "jump there". Writing that by hand is possible but painfully slow. Programming languages let you write `total = rock_mass * 0.35` instead of dozens of numeric instructions. Something else then does the translation.

Two ways of translating

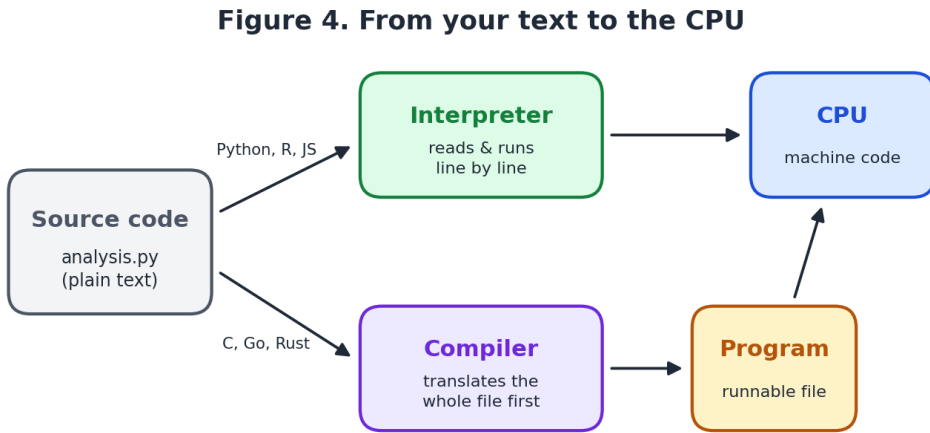


Figure 4. Source code goes either through an interpreter line by line, or through a compiler into a runnable program, before reaching the CPU

- **Compiled** (C, C++, Go, Rust): a **compiler** translates the whole program into machine code first, producing a runnable file. It is like translating a whole paper before publishing it. There is an extra build step, errors show up early, and the result runs fast.
- **Interpreted** (Python, JavaScript, R): an **interpreter** reads your code and carries it out line by line. It is like a live interpreter at a conference. You can run code immediately, but an error only appears when that line is reached, and it runs more slowly.

Java sits in between: it compiles to an intermediate form that a virtual machine then runs. You do not need to worry about that detail yet.

The practical consequence: **when you "install Python", you are installing the interpreter.** Your `.py` file is just text. The `python` program is what reads it and makes it happen. No interpreter, no running code.

PLAYGROUND Step through three small programs one line at a time and watch each variable appear as a labelled box on the bench: [Chapter 6 playground](#).

Source code, and where it runs

- **Source code** is the text you write, saved in files like `analysis.py`.
- An **editor** or **IDE** (such as VS Code, PyCharm or IDLE) is a program for writing that text comfortably. It is not the language. VS Code with no Python installed cannot run Python.
- The code runs **on a specific machine**, under that machine's OS, reading files from that machine's storage. When code "works on my laptop but not yours", the difference is almost always in that environment, not in the code.

Every language has the same core ideas

Once you see these, learning a second language becomes much easier. Each language just spells them differently.

IDEA	WHAT IT IS	GEOLOGY EXAMPLE
Variable	A named box holding a value	<code>porosity = 0.23</code>
Data type	What kind of value it is: number, text, true/false	"granite" is text; 2.65 is a number
Collection	Many values together: a list or table	A list of 50 borehole depths
Condition	Do something only if a test is true	If silica > 63%, label it felsic
Loop	Repeat an action	For each sample, compute density
Function	A named, reusable procedure with inputs and outputs	<code>density(mass, volume)</code>
Input / output	Reading and writing files, screen, network	Read <code>boreholes.csv</code> , write <code>results.csv</code>

The hard part of programming is rarely the syntax. It is breaking a problem into these small, exact steps. You already do this when you design a sampling or analysis protocol.

Scripts and notebooks: two ways to write Python

So far, code lives in a **script**: a `.py` file you run from start to finish with `python analysis.py`. Scientists often use a second format, the **Jupyter notebook** (a `.ipynb` file).

A notebook is split into **cells**. Some cells hold code, others hold notes written in plain text. You run one cell at a time, and its output, whether a number, a table or a plot, appears directly underneath. It works like a lab notebook where each observation sits next to the procedure that produced it.

Behind every notebook is a **kernel**: a running Python process that remembers everything the cells have done so far. This makes notebooks great for exploring data step by step. It also causes their most common confusion: if you run cells out of order, or delete a cell, the kernel still remembers the old values. When a notebook behaves strangely, use **Restart Kernel and Run All** to replay it cleanly from the top.

USE A SCRIPT WHEN...	USE A NOTEBOOK WHEN...
The job runs start to finish	You are exploring or plotting data
You will run it repeatedly	You want notes, code and results together
It is an MCA assignment asking for a <code>.py</code> file	You are following Earth Lab or Project Pythia

To make a notebook in VS Code, create a file ending in `.ipynb`, choose your Python as the kernel when asked, type code in a cell and press **Shift + Enter** to run it.

Try it

If Python is installed, open a terminal and type `python` (or `python3`) and press Enter. You are now talking to the interpreter directly. Type `2.65 * 1000` and press Enter. Then `rock = "basalt"` and then `rock.upper()`. Type `exit()` to leave. You just ran code without any file at all.

■ Watch alongside

- [Crash Course CS #11: The First Programming Languages \(12 min\)](#)
- [Crash Course CS #12: Programming Basics: Statements and Functions \(12 min\)](#)
- [Harvard CS50P, Lecture 0: Functions, Variables \(free course; start with lecture 0, then 1 Conditionals and 2 Loops\)](#)

ON THE COMPANION SITE Chapter 6 on the companion site

CHAPTER 7

Libraries, packages and package managers

A **library** is code someone else already wrote, packaged so you can reuse it. Almost no real program is written from scratch. Programmers spend much of their time finding, installing and combining libraries, then writing a small amount of their own code to glue them together.

The reagent analogy

When you ran an XRF or ICP-MS analysis, you did not synthesise your own acids or build the spectrometer. You ordered standard reagents from a supplier, checked the grade and batch, and used them in your procedure. Your contribution was the method and the interpretation.

Libraries are the reagents and instruments of programming:

- **Library / package:** the reagent. A bundle of ready-made functions. ("Package" usually means a library bundled for installation. People use the words almost interchangeably.)
- **Package registry:** the supplier's catalogue. For Python it is **PyPI** (pypi.org); for JavaScript it is **npm**.
- **Package manager:** the ordering system that fetches the package, checks what else it needs, and installs it. For Python this is **pip**.

- **Version:** the batch number. Version 2.1 and version 3.0 of a library can behave differently.
- **Dependencies:** reagents that your reagent needs. Installing one package often quietly installs ten others.

The standard library vs. installed packages

Every language ships with a **standard library**: basic tools that come with the interpreter, like the glassware that comes with the lab. In Python, `math`, `csv` and `os` are always there.

Everything else must be **installed** first. For geoscience work in Python, the common ones are:

PACKAGE	WHAT IT IS FOR
<code>numpy</code>	Fast maths on arrays of numbers
<code>pandas</code>	Tables of data: reading CSV/Excel, filtering, grouping
<code>matplotlib</code>	Plots and charts
<code>scipy</code>	Statistics, interpolation, signal processing
<code>geopandas</code>	Maps and spatial data (shapefiles, GeoJSON)
<code>rasterio</code>	Raster data such as DEMs and satellite images

The two steps: install, then import

This is the source of most beginner confusion, because the two steps happen in **different places**.

1. **Install** (once, in the terminal): download the package onto your computer.

```
pip install pandas
```

2. **Import** (every time, inside your code): tell *this program* you want to use it.

```
import pandas as pd
data = pd.read_csv("boreholes.csv")
```

Figure 5. Install once, import every time

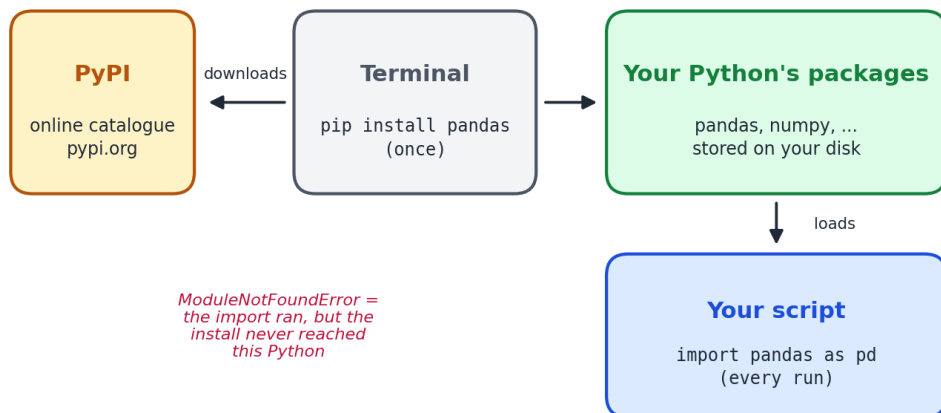


Figure 5. Pip downloads a package from PyPI into your Python's packages folder once; your script imports it every run

Installing is ordering the reagent into the store room. Importing is taking it to your bench for today's procedure. If you see `ModuleNotFoundError: No module named 'pandas'`, the import happened but the install never did, or it was installed for a *different* Python (Chapter 8).

PLAYGROUND A laptop with two Pythons. Install pandas with plain pip, watch the import still fail, then fix it with `python -m pip`: [Chapter 7 playground](#).

`pip install` is typed in the terminal, never inside a `.py` file or the Python `>>>` prompt. Typing it in the wrong place gives a `SyntaxError`.

How to judge a package

Anyone can publish a package, just as anyone can sell chemicals. Before using one, check:

- Is it widely used? (Download counts, GitHub stars, mentions in tutorials.)
- Is it maintained? (Recent releases, recent activity.)
- Does it have documentation with examples?
- Is the name spelled exactly right? Fake look-alike packages exist.

Try it

In the terminal, run `pip install pandas` (or `pip3 install pandas`). Watch it download pandas *and* its dependencies. Then start Python, type `import pandas`, and press Enter. No error means the reagent is on your shelf.

Watch alongside

- [Harvard CS50P, Lecture 4: Libraries](#) (explains `import`, the standard library and installing packages with `pip`)
- [Corey Schafer: Python Tutorial: pip](#) (about 15 min, practical walkthrough)

ON THE COMPANION SITE [Chapter 7 on the companion site](#)

CHAPTER 8

Versions and environments

The same code can behave differently on two computers because the **environment** differs: which interpreter version, which packages, which versions of those packages, which OS. Controlling the environment is how programmers make results reproducible, the same way you would control conditions in an experiment.

Version numbers

Most software uses three numbers: `MAJOR.MINOR.PATCH`, for example `pandas 2.2.1`.

- **PATCH** (2.2.1): bug fixes only. Safe to update.
- **MINOR** (2.2.1): new features, old code should still work.
- **MAJOR** (2.2.1): breaking changes. Code written for version 1 may fail on version 2.

This matters when following tutorials. A tutorial from 2019 may use functions that no longer exist. When code from the internet fails in a strange way, check the date and the version it was written for.

"Which Python is this?"

Many computers end up with more than one Python installed, for example one from `python.org`, one from Anaconda, one bundled with another tool.

Each has its **own** set of installed packages. This leads to the most frustrating beginner problem: you ran `pip install pandas`, it succeeded, but your code still says pandas is missing. The package went to one Python, and your code ran with another.

Two commands help you check:

```
python --version
pip --version
```

The second one shows which Python that pip belongs to. They should match. A safe habit is to install with `python -m pip install pandas`, which guarantees pip installs into the same Python you are running.

Virtual environments

A **virtual environment** is a separate, self-contained set of packages for one project. It is like giving each research project its own sealed sample box, so contamination from one cannot affect another.

Why bother? Project A may need `numpy 1.x` while Project B needs `numpy 2.x`. Without separate environments, installing one breaks the other.

Figure 6. One sealed box of packages per project

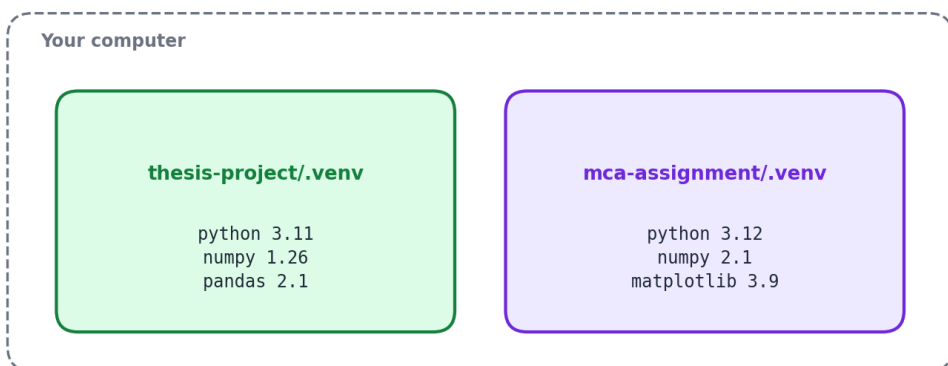


Figure 6. Two projects on one computer, each with its own virtual environment and package versions

The basic Python workflow:

```
python -m venv .venv          # create the environment (once per
project)
.venv\Scripts\activate        # Windows: switch it on
source .venv/bin/activate     # macOS/Linux: switch it on
python -m pip install pandas  # now installs only into this project
deactivate                   # switch it off
```

If Windows refuses to run the activate script with a message about "execution policies", run this once in PowerShell and try again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

When an environment is active, its name usually appears in brackets at the start of your terminal prompt, such as `(.venv)`.

PLAYGROUND Create, activate and fill a virtual environment, execution-policy error included: [Chapter 8 playground](#).

Recording the environment

Projects usually list their packages in a file named `requirements.txt`, which works like the materials section of a methods paper. Anyone can recreate your setup with:

```
pip install -r requirements.txt
```

Anaconda (conda) is another popular tool for the same job. It is common in scientific computing because it also handles non-Python libraries that geospatial packages depend on. Either approach is fine; just be consistent within a project.

Try it

Make a new folder for practice, `cd` into it, and create and activate a virtual environment using the commands above. Run `pip list` to see how empty it is. Install one package and run `pip list` again.

■ Watch alongside

- [Corey Schafer: VENV \(Windows\), How to Use Virtual Environments or the Mac and Linux version](#)
- [MIT Missing Semester, Lecture 8: Metaprogramming](#) (covers dependency management and versions; watch later)

ON THE COMPANION SITE Chapter 8 on the companion site

CHAPTER 9

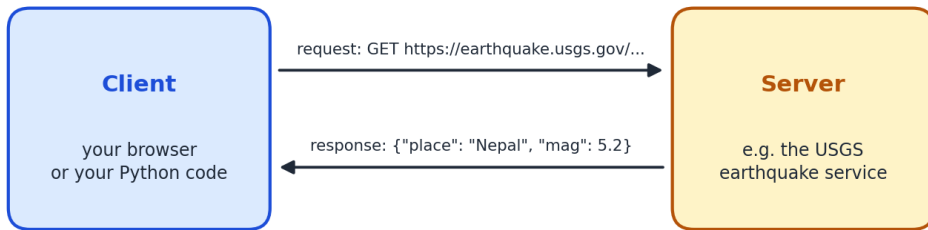
Networks, the internet and how programs talk

The internet is a huge network of computers that pass messages to each other using agreed rules called **protocols**. When you open a website, your computer sends a request message to another computer and gets a response back. Almost everything online, including apps, maps, and data downloads, is this same request-and-response exchange.

Clients and servers

- A **client** asks for something. Your browser is a client.
- A **server** is a program (and the computer it runs on) that waits for requests and answers them.

It is like requesting a dataset from the Geological Survey. You send a request form (client). Their data office (server) looks it up and sends a reply. "The server is down" means the data office is closed; your request has nowhere to go.

Figure 7. Request and response

DNS first turns the name `usgs.gov` into an IP address, like a phone directory.

Figure 7. A client sends a request to a server and receives a JSON response

Addresses

- Every computer on a network has an **IP address**, a number like `142.250.183.46`, like a postal address.
- Humans use **domain names** like `usgs.gov`. A system called **DNS** translates names into IP addresses, like a phone directory.
- A **URL** is a full address for one resource: `https://earthquake.usgs.gov/earthquakes/feed/` gives the protocol (`https`), the domain, and the path to the specific item. Notice the path looks just like a file path from Chapter 3.

`https` means the conversation is encrypted, so people in between cannot read it. `http` without the `s` is not encrypted.

APIs: how programs ask each other for things

An **API** (Application Programming Interface) is a set of requests one program agrees to accept from another. A website is designed for humans to read. An API is designed for code to read.

Many scientific data sources offer APIs. For example, earthquake catalogues, satellite imagery portals and weather services let you request data by URL and receive it as structured text, usually **JSON**:

```
{"place": "Nepal", "magnitude": 5.2, "depth_km": 10}
```

Your code can then read that JSON directly, with no manual downloading or copying from web pages.

The word "API" is also used more broadly for *any* defined way to use a piece of code. A library's API is the list of functions it offers you. When documentation says "see the API reference", it means that list.

PLAYGROUND Follow a request to the USGS and back, then click through a saved JSON snapshot to see the Python that reaches each value: [Chapter 9 playground](#).

Local vs. remote

A distinction that confuses many beginners:

- **Local** means on your own computer. `localhost` (or `127.0.0.1`) is how a computer refers to itself.
- **Remote** means on another computer, such as a server, a university cluster, or "the cloud" (which is simply someone else's computers that you rent).

When a tutorial says "open `localhost:8000` in your browser", it means a server is running on your own machine, and your browser is acting as its client.

■ Try it

Paste this URL into your browser: `https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/significant_month.geojson`. The page you see is raw JSON from an API, meant for code, not people. Find one earthquake's place and magnitude in it. Later, you will fetch this same data with three lines of Python.

■ Watch alongside

- Crash Course CS #28: Computer Networks (12 min)
- Crash Course CS #29: The Internet (12 min)
- Crash Course CS #30: The World Wide Web (11 min)

ON THE COMPANION SITE Chapter 9 on the companion site

CHAPTER 10

Errors, and how to find answers

Professional programmers do not know everything. They are good at **reading errors and searching well**. They look things up dozens of times a day, and nobody considers that cheating. It is the job, just as consulting the literature is part of research.

Errors are data, not failure

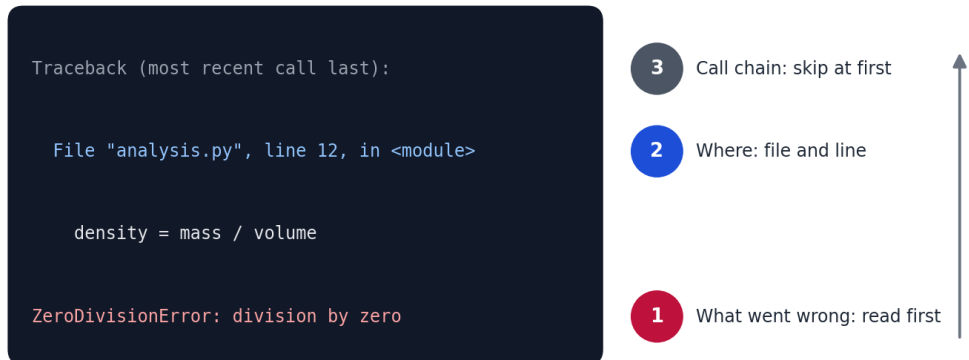
An error message is the computer telling you exactly what went wrong and where. Beginners tend to panic and skip it. Instead, read it like a field observation.

A Python error (called a **traceback**) looks like this:

```
Traceback (most recent call last):
  File "analysis.py", line 12, in <module>
    density = mass / volume
ZeroDivisionError: division by zero
```

Read it **from the bottom up**:

1. **Last line**: the type of error and a short description. This is the most important line.
2. **Lines above it**: where it happened, with the file name and line number.
3. **Further up**: the chain of calls that led there. Useful later; ignore it at first.

Figure 8. Read a traceback from the bottom up**Figure 8.** A Python traceback annotated to be read from the bottom line upwards

PLAYGROUND Four traceback puzzles: click the line that names the problem, then pick the fix: [Chapter 10 playground](#).

Some errors you will meet constantly:

ERROR	USUALLY MEANS
<code>SyntaxError</code>	You wrote something the language cannot parse: missing bracket, colon or quote
<code>NameError</code>	You used a name that was never defined, often a typo
<code>TypeError</code>	You combined incompatible kinds of value, such as adding text to a number
<code>FileNotFoundError</code>	The path is wrong, or you are running from a different folder (Chapter 3)
<code>ModuleNotFoundError</code>	The package is not installed in this Python (Chapters 7–8)
<code>IndentationError</code>	Spaces at the start of lines are inconsistent (Python cares about these)

How to search: a method

1. **Copy the last line of the error exactly.** Paste it into a search engine.
2. **Add the language and key library.** For example: `python pandas KeyError: 'depth'`.
3. **Remove your own specific names** if results are poor. Your column name `depth` is unique to you; the error type is not. Try `pandas KeyError column`.
4. **Describe the goal, not the code**, when you do not know what to do at all: `python read excel file specific sheet, pandas group by and average`.
5. **Check the date and version.** An answer from 2015 may be outdated (Chapter 8).

Searching for the *goal* is a skill worth practising. Almost every small task ("sort a table by a column", "plot two lines on one chart", "convert degrees to radians") has been asked and answered before.

Where good answers live

- **Official documentation** gives the definitive meaning of a function and its options. It can be dense, so jump to the "Examples" and "Getting started" sections.
- **Stack Overflow** is best for specific errors and "how do I..." questions. Prefer the accepted or highest-voted answer, and read the comments for updates.
- **Tutorials and blogs** are good for learning a topic end to end. Quality varies, so check the date.
- **AI assistants** can explain an error, a concept or a block of code. They can be confidently wrong, so test what they give you and ask them to explain, not just to fix.

Reading documentation is like reading a methods section. You do not read it cover to cover. You go straight to the function you need, read its inputs and outputs, then look at the example.

Asking a person for help

When you ask a friend, a forum or a teacher, include:

- What you were **trying** to do
- The **exact** code (the smallest piece that shows the problem)
- The **full** error message, copied as text, not a phone photo of the screen
- What you already **tried**

Writing this out often reveals the answer before you even send it.

Try it

In Python, type `"basalt" + 5` and press Enter. Read the error bottom-up. Copy the last line into a search engine with the word `python`, open the first Stack Overflow result, and find the explanation. Then fix it yourself.

Watch alongside

- [Harvard CS50P, Lecture 3: Exceptions](#) (what errors are and how to handle them)
- [MIT Missing Semester, Lecture 7: Debugging and Profiling](#) (watch after a few weeks of coding)

ON THE COMPANION SITE Chapter 10 on the companion site

CHAPTER 11

How to learn a language without getting lost

The feeling of "I understand each line but not the point" comes from learning parts without a purpose to attach them to. The fix is to **learn through small, real problems** from a field you already understand, and to learn only what each problem needs.

Principles

- **One language first.** For MCA coursework you may need C, Java or others, but pick one (Python is the gentlest) to build intuition. Concepts from Chapter 6 then transfer; only the spelling changes.
- **Type, don't copy.** Typing code, even from an example, forces you to notice each part. Copy-pasting teaches nothing.
- **Change things and predict.** Before running a modified line, predict what will happen. When you are wrong, you have learned something specific.
- **Small steps, run often.** Write two or three lines, run them, check the result. Never write fifty lines and then run them for the first time.
- **Print everything.** When confused about what a variable holds, `print()` it. This is the programming equivalent of looking at the sample under the microscope rather than guessing.

- **Being stuck is normal.** Spend 15–20 minutes trying (Chapter 10), then ask. Struggling a little builds understanding; struggling for hours just builds frustration.

PLAYGROUND Five snippets to predict before you reveal what they print: [Chapter 11 playground](#).

A practice path using geology

Each step adds a few concepts. Do them in order, and use real data from your PhD where you can.

1. **Calculator scripts.** Compute density from mass and volume; convert units. (*Variables, numbers, arithmetic, print.*)
2. **Classify one sample.** Given silica %, print felsic, intermediate, mafic or ultramafic. (*Conditions.*)
3. **Classify many samples.** Store 10 silica values in a list and classify each. (*Lists, loops.*)
4. **Make it reusable.** Turn the classifier into a function `classify(silica)`. (*Functions.*)
5. **Read a real file.** Load a CSV of your own sample data and classify every row. (*Files, paths, the `csv` module or `pandas`.*)
6. **Summarise.** Count how many of each rock type; average values per type. (*`pandas` grouping.*)
7. **Plot.** Draw a histogram or a scatter plot, such as a simple TAS-style diagram. (*`matplotlib`.*)
8. **Fetch live data.** Download recent earthquakes from the USGS API (Chapter 9) and plot magnitude against depth. (*Packages, APIs, JSON.*)

By step 8, every chapter of this book will have appeared in your own work.

Using your editor well

Install **VS Code** (free) with its Python extension. It colours your code, underlines mistakes before you run it, and shows a function's documentation

when you hover over it. Keep one folder per project, and open that folder in VS Code so the working directory is always predictable.

A weekly rhythm

- A few short sessions (30–60 minutes) beat one long weekend session.
- End each session by writing one line in a notes file: what you learned and what confused you. Next session, start from the confusion.
- Every few weeks, re-read a chapter of this book. It will mean more each time.

■ Watch alongside

- [Harvard CS50P, the full course](#): lectures 0–6 match practice steps 1–5 above; Lecture 6 (File I/O) covers reading CSV files
- [Crash Course CS #13: Intro to Algorithms](#) (12 min)
- [Crash Course CS #16: Software Engineering](#) (11 min)
- For steps 5–8, work through [Earth Lab's Earth Data Science lessons](#) and [Project Pythia](#), which use Earth science data throughout

ON THE COMPANION SITE Chapter 11 on the companion site

CHAPTER 12

Saving your history with Git

Git records snapshots of a project folder over time, so you can see what changed, when and why, and go back to any earlier version. It replaces the familiar mess of `thesis_final.docx`, `thesis_final2.docx` and `thesis_REALLY_final.docx`. Almost every programming project in the world uses it.

The field notebook analogy

A good field notebook has dated entries: what you did, what you found, and why. You never tear out pages; you add new entries. Git is a field notebook for a folder.

- A **repository** (repo) is a folder whose history Git is tracking.
- A **commit** is one dated entry: a snapshot of chosen files plus a short message saying why.
- The **log** is the list of all commits, newest first.
- **Staging** is choosing which changes go into the next entry, like deciding which samples to record before writing up.
- **GitHub** is a website that stores a copy of your repository online, for backup and sharing. Git works fine without it.

Install and introduce yourself

Download Git from git-scm.com and install it with the default options. Then, in a new PowerShell window, tell Git who you are (once per computer):

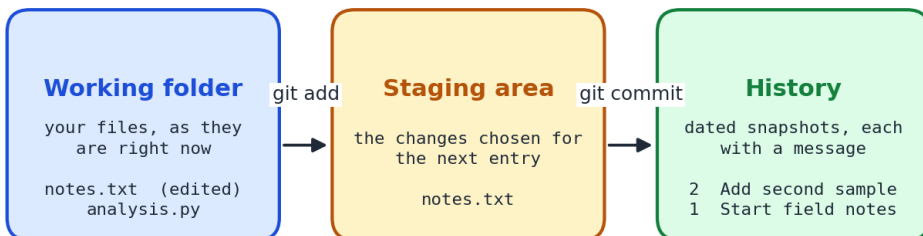
```
git config --global user.name "Neha"
git config --global user.email "you@example.com"
```

The everyday cycle

```
cd C:\Users\Neha\Documents\learning
git init                    # start tracking this folder (once)
git status                  # what has changed?
git add hello.py            # stage a file for the next commit
git commit -m "First program" # save the snapshot with a message
git log --oneline           # see the history
```

From then on, the rhythm is: edit files → `git status` → `git add` → `git commit -m "why"`. Commit whenever you reach a small working step. Write messages that explain *why*, such as "Classify samples by silica", not "changes".

Git's three trays: working folder, staging area, history



`git status` reports what is in each tray. `git log` lists the history, newest first.

Figure 10. Git's three trays, the working folder, the staging area and the history, with `git add` and `git commit` moving changes between them

PLAYGROUND A repository with three live trays. Every Git command you type shows up in the tray it moves things to: [Chapter 12 playground](#).

Things Git should ignore

Some files should never be committed: virtual environments, very large data files, and passwords. List them in a plain-text file named `.gitignore` in the repository folder:

```
.venv/  
*.tif  
.env
```

Using Git from VS Code

VS Code has Git built in. The **Source Control** icon on the left (a branching line) shows changed files. Type a message in the box and click **Commit**. It runs exactly the commands above, so knowing them helps you understand what the button does.

Try it

In your `learning` folder, run `git init`, then commit `hello.py`. Change the message in `hello.py`, run `git status` to see Git notice the change, and make a second commit. Finally run `git log --oneline` and count your commits.

Watch alongside

- [MIT Missing Semester, Lecture 6: Version Control \(git\)](#) (about 85 min; the first 30 minutes are enough to start)
- Practise with the [Software Carpentry Version Control with Git lesson](#), written for researchers
- Read chapters 1–2 of [Pro Git](#) (free)

ON THE COMPANION SITE Chapter 12 on the companion site

CHAPTER 13

Capstone — one geology project, end to end

This chapter builds one small project from an empty folder to a finished plot, and uses almost every idea in the book along the way. You will read a table of rock samples, classify each sample by silica content, summarise the classes, plot them, save the results, and record the work in Git. Type the code yourself rather than copying it (Chapter 11).

The data below is invented practice data. Once it works, run the same script on real samples from your own PhD work.

Step 1: Set up the project (Chapters 3, 5, 8, 12)

In PowerShell:

```
cd C:\Users\Neha\Documents\learning
mkdir capstone
cd capstone
python -m venv .venv
.venv\Scripts\activate
python -m pip install pandas matplotlib requests
git init
```

Then open the `capstone` folder in VS Code (*File* → *Open Folder*). You now have a project folder, a private set of packages, and an empty history.

Step 2: Create the data file (Chapter 3)

In VS Code, create `samples.csv` in the `capstone` folder and paste in this table. SiO₂ and MgO are in weight percent; density is in g/cm³.

```
sample_id,location,sio2,mgo,density
NH-01,Ridge A,48.2,8.9,2.95
NH-02,Ridge A,50.1,7.4,2.91
NH-03,Ridge A,55.6,4.1,2.78
NH-04,Valley B,61.8,2.6,2.72
NH-05,Valley B,68.4,1.1,2.66
NH-06,Valley B,72.9,0.4,2.63
NH-07,Quarry C,43.1,31.5,3.25
NH-08,Quarry C,41.7,36.2,3.30
NH-09,Quarry C,51.3,6.8,2.89
NH-10,Ridge A,58.9,3.3,2.75
NH-11,Valley B,70.2,0.7,2.64
NH-12,Quarry C,46.5,9.8,2.97
```

The first line is the **header**: column names. Every later line is one sample. Commas separate the values. It is plain text (Chapter 3), so any editor can open it.

Step 3: Read the data (Chapters 6, 7)

Create `analysis.py` and start with:

```
import pandas as pd
import matplotlib.pyplot as plt

samples = pd.read_csv("samples.csv")
print(samples.head())
print("Rows and columns:", samples.shape)
```

- `import pandas as pd` loads the installed pandas package and gives it the short name `pd`.
- `pd.read_csv("samples.csv")` reads the file into a **DataFrame**, pandas' name for a table. The path is **relative**, so this works only when you run from the `capstone` folder.
- `samples.head()` shows the first five rows; `samples.shape` gives (rows, columns).

Run `python analysis.py`. You should see five rows, then `Rows` and `columns:` `(12, 5)`.

Step 4: Classify one value (Chapter 6)

Add a **function** that turns a silica value into a rock class, using the standard SiO_2 boundaries:

```
def classify(sio2):
    if sio2 < 45:
        return "ultramafic"
    elif sio2 < 52:
        return "mafic"
    elif sio2 < 63:
        return "intermediate"
    else:
        return "felsic"
```

`def` defines the function; `sio2` is its input. The `if` / `elif` / `else` **conditions** are checked from top to bottom, and the first true one decides the answer. Indentation (four spaces) shows which lines belong inside the function and inside each condition.

Step 5: Classify every sample (Chapters 6, 7)

```
samples["rock_class"] = samples["sio2"].apply(classify)
print(samples[["sample_id", "sio2", "rock_class"]])
```

`samples["sio2"]` picks one column. `.apply(classify)` runs your function on every value in it, which is a **loop** without writing the loop yourself. The results go into a new column called `rock_class`.

Step 6: Summarise (Chapter 7)

```
summary = samples.groupby("rock_class").agg(
    count=("sample_id", "count"),
    mean_density=("density", "mean"),
)
print(summary)
```

`groupby` sorts rows into piles by rock class, like sorting hand specimens into trays. `agg` then counts each pile and averages its density. Check that the result makes geological sense: density should rise from felsic to ultramafic.

Step 7: Plot and save (Chapters 3, 7)

```
fig, ax = plt.subplots()
for name, group in samples.groupby("rock_class"):
    ax.scatter(group["sio2"], group["mgo"], label=name)
ax.set_xlabel("SiO2 (wt%)")
ax.set_ylabel("MgO (wt%)")
ax.set_title("Neha's samples: MgO against SiO2")
ax.legend()
fig.savefig("sio2_vs_mgo.png", dpi=150)

samples.to_csv("results.csv", index=False)
print("Saved results.csv and sio2_vs_mgo.png")
```

The `for` loop draws each rock class in its own colour. `savefig` and `to_csv` write new files into the working directory; open the `capstone` folder and you will find them there.

Step 8: Record the work (Chapter 12)

Create a `.gitignore` file containing `.venv/`, then:

```
git add .gitignore samples.csv analysis.py
git commit -m "Classify samples by silica and plot MgO against SiO2"
```

The outputs (`results.csv`, the PNG) can always be regenerated by running the script, so many people leave them out of Git.

Step 9 (extension): live earthquake data (Chapter 9)

Create `quakes.py`. It asks the USGS API for every magnitude 4.5+ earthquake from the past month and plots magnitude against depth:

```
import requests
import pandas as pd
import matplotlib.pyplot as plt

url = "https://earthquake.usgs.gov/earthquakes/feed/v1.0/summary/4.5_month.geojson"
data = requests.get(url, timeout=30).json()

rows = []
for feature in data["features"]:
    lon, lat, depth = feature["geometry"]["coordinates"]
    rows.append({
        "place": feature["properties"]["place"],
        "magnitude": feature["properties"]["mag"],
        "depth_km": depth,
    })

quakes = pd.DataFrame(rows)
print(len(quakes), "earthquakes")
print(quakes.sort_values("magnitude", ascending=False).head())

fig, ax = plt.subplots()
ax.scatter(quakes["depth_km"], quakes["magnitude"], alpha=0.5)
ax.set_xlabel("Depth (km)")
ax.set_ylabel("Magnitude")
fig.savefig("quakes.png", dpi=150)
```

`requests.get` is the **client** sending a request (Figure 7). `.json()` turns the JSON reply into Python lists and dictionaries. The loop picks out three values from each earthquake and builds a table. Because the data is live, your numbers will differ every time you run it.

Try it

Change the boundaries so that anything at or above 63% SiO₂ is split further: 63–69% "intermediate-felsic" and above 69% "felsic". Run the script again, check the new counts, and commit the change with a clear message.

■ Watch alongside

- Harvard CS50P, Lecture 6: File I/O (reading and writing files, including CSV)
- Corey Schafer: Pandas Tutorial (Part 1): Getting Started with Data Analysis and the next few parts
- Corey Schafer: Matplotlib Tutorial (Part 7): Scatter Plots

ON THE COMPANION SITE Chapter 13 on the companion site

Afterword: your own project

The capstone used invented rocks. The next step is the same project with data you care about, in your own folder, on your own machine. Here is the capstone as a template; the blanks are yours to fill.

1. **Name the question.** Write one sentence: *I want to know _____ about _____*. If you cannot write it, the analysis is not ready yet.
2. **Get the data into a CSV.** One row per sample, site or measurement; one column per variable; a header row; no merged cells, no colour coding, no totals row. From Excel, use *File* → *Save As* → CSV.
3. **Make the folder.** `C:\Users\Neha\Documents\projects\_____` containing `data\` for the CSV, `analysis.py`, a `.venv`, a `.gitignore` with `.venv/` in it, and later `requirements.txt`.
4. **Read it and look.** `print(samples.shape)`, `print(samples.head())`, `print(samples.dtypes)`. A column that should be numbers but shows `object` has a stray text value somewhere; find it before going on.
5. **Compute one thing.** A function, like `classify`, that turns one row into one answer. Test it with three values whose answers you already know.
6. **Apply, group, summarise, plot.** `apply`, `groupby`, `agg`, one figure saved to a PNG.
7. **Commit** the CSV and the code, not the outputs.
8. **Write a README.md:** the question, where the data came from, and the one command that runs the analysis.

Three projects that fit this template and your own work:

- **Borehole logs.** One row per depth interval with its lithology. Question: how thick is each lithology in each hole, and which hole has the most of _____?
- **Sample geochemistry.** Your own major-element analyses. Question: how do the samples classify, and does the classification agree with what you named them in hand specimen?
- **Earthquake catalogue.** A month of events from the USGS API (Chapter 9). Question: how does depth relate to magnitude, and does the relationship differ by region?

When the first one works, you are no longer doing exercises.

Appendix: What you should see

Check your "Try it" results here *after* attempting each one. Your exact numbers may differ from these; what matters is the shape of the result.

Chapter 1. A processor name (for example *Intel Core i5* or *Apple M2*), RAM of roughly 8–32 GB, and storage of roughly 256 GB–1 TB. Storage is always much larger than RAM, just as an archive is larger than a bench.

Chapter 2. A long list of processes, far more than the programs you opened, because the operating system runs many background processes. Opening a browser tab usually adds one or more new entries.

Chapter 3. A path such as `C:\Users\Neha\Documents\notes.docx`. Reading it from the left: drive root, then the Users folder, your folder, Documents, and finally the file.

Chapter 4. `pwd` prints your current folder, usually `C:\Users\Neha` when PowerShell first opens. After `cd` into a folder, `pwd` shows that folder added on the end. After `cd ..` you are back where you started. Your drawing should match the tree in your file explorer.

Chapter 5. Your new message prints in the terminal. `python --version` shows `Python 3.` followed by two more numbers, such as `Python 3.12.6`. If you see an error instead, use the "If something goes wrong" table.

Chapter 6. `2.65 * 1000` gives `2650.0`. `rock.upper()` gives `'BASALT'`. The prompt `>>>` means the interpreter is waiting for your next line.

Chapter 7. `pip` downloads pandas plus several dependencies, such as numpy and python-dateutil, then ends with "Successfully installed...". `import pandas` prints nothing at all. In Python, silence after an import means success.

Chapter 8. Right after creating the environment, `pip list` shows only pip itself (and perhaps setuptools). After installing one package, it shows that package plus its dependencies. Your prompt starts with `(.venv)` while the environment is active.

Chapter 9. The browser shows raw text starting with

```
{"type": "FeatureCollection"}. Each earthquake has a "place" such as "120 km S of ..."
```

 and a "mag" number. The list changes daily.

Chapter 10. The last line reads `TypeError: can only concatenate str (not "int") to str`. Python will not join text and a number. Fix it with `"basalt" + str(5)` to get `'basalt5'`, or `"basalt" * 5` if you wanted repetition.

Chapter 11. No single answer. Your notes file should have one line per session, recording what you learned and what confused you.

Chapter 12. `git status` shows `hello.py` as *modified* in red after your edit. `git log --oneline` lists two commits, newest first, each with a short code and your message.

Chapter 13. With the practice data, the original run gives 3 felsic, 3 intermediate, 4 mafic and 2 ultramafic samples. Mean densities rise from about 2.64 g/cm³ (felsic) to about 3.28 g/cm³ (ultramafic). After your change, the 3 felsic samples split into 1 intermediate-felsic (NH-05, 68.4%) and 2 felsic (NH-06 and NH-11).

Cheat sheet: the commands you will keep looking up

PowerShell (Chapter 4)

COMMAND	WHAT IT DOES
<code>pwd</code>	Show the folder you are in
<code>ls</code>	List what is here
<code>cd folder</code> · <code>cd ..</code> · <code>cd \$HOME</code>	Go into a folder · up one level · home
<code>mkdir name</code>	Make a folder
<code>echo text > file.txt</code> · <code>echo text >> file.txt</code>	Create or overwrite a file · add a line to it
<code>cat file.txt</code>	Show a file's contents
<code>cp a b</code> · <code>mv a b</code> · <code>rm a</code>	Copy · move or rename · delete (no Recycle Bin)
<code>Tab</code> · <code>↑</code> · <code>Ctrl + C</code>	Complete a name · repeat an earlier command · stop a command

Python and pip (Chapters 5-7)

COMMAND	WHAT IT DOES
<code>python --version</code>	Which Python answers to <code>python</code>
<code>python script.py</code>	Run a script from the folder you are in

COMMAND	WHAT IT DOES
<code>python -c "print(2 + 2)"</code>	Run one line without making a file
<code>python ... exit()</code>	Start and leave the interactive <code>>>></code> prompt
<code>python -m pip install pkg</code>	Install a package into this Python
<code>python -m pip show pkg · python -m pip list</code>	Details of one package · all of them
<code>python -m pip freeze > requirements.txt</code>	Record the installed packages
<code>python -m pip install -r requirements.txt</code>	Install from a recorded list

Virtual environments (Chapter 8)

COMMAND	WHAT IT DOES
<code>python -m venv .venv</code>	Create an environment in this folder
<code>.venv\Scripts\activate</code>	Turn it on; the prompt shows <code>(.venv)</code>
<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>	Once, if Windows refuses to run the activate script
<code>deactivate</code>	Turn it off
<code>source .venv/bin/activate</code>	The Mac and Linux activate

Git (Chapter 12)

COMMAND	WHAT IT DOES
<code>git config --global user.name "Neha" · user.email "..."</code>	Once, before the first commit
<code>git init</code>	Start tracking this folder
<code>git status</code>	What is changed, staged and untracked

COMMAND	WHAT IT DOES
<code>git add file · git add .</code>	Stage one file · everything
<code>git commit -m "Why this change"</code>	Save a snapshot with a message
<code>git log --oneline</code>	The history, newest first
<code>.gitignore</code> containing <code>.venv/</code>	Keep the environment out of Git

Reading an error (Chapter 10). Last line first. `ModuleNotFoundError`: install it into this Python. `FileNotFoundError`: which folder am I running from? `NameError`: a spelling mismatch. `TypeError` with `str`: text and a number joined with `+`. `SyntaxError`: look at the line above the arrow, often a missing colon or bracket.

Glossary

TERM	PLAIN MEANING	CHAPTER
API	A defined set of requests a program or library accepts from other code	9
Absolute path	A file address starting from the root	3
Argument	Something passed to a command or function to act on	4, 6
Binary	Numbers written with only 0 and 1; how computers store everything	1
Bug	A mistake in code that makes it behave wrongly	10
Cell	One block of code or notes in a Jupyter notebook	6
Client / server	The side that asks / the side that answers	9
Commit	A saved snapshot in Git, with a message saying why	12
Compiler	Translates a whole program into machine code before running	6
CPU	The processor that carries out instructions	1
CSV	A table saved as plain text with comma-separated values	3
DataFrame	pandas' name for a table of data	13
Dependency	A package that another package needs	7

TERM	PLAIN MEANING	CHAPTER
Directory	Another word for folder	3
Documentation (docs)	The official explanation of how software or a library works	10
Environment	The interpreter, packages and versions your code runs with	8
Function	A named, reusable block of code with inputs and outputs	6
Git	A tool that records the history of a project folder	12
IDE / editor	A program for writing code, such as VS Code	5, 6
Import	Load an installed package into your program	7
Interpreter	Runs code line by line; Python is one	6
JSON	A plain-text format for structured data	9
Jupyter notebook	A file of code and notes run cell by cell, with results shown inline	6
Kernel	The running Python process behind a notebook	6
Library / package	Reusable code written by others	7
Localhost	Your own computer, as seen by itself on a network	9
Operating system	Software that manages hardware and runs other programs	2
PATH	The OS list of folders searched for commands	4, 5
pip	Python's package manager	7
Process	A program that is currently running	2
RAM	Fast, temporary working memory	1
Relative path	A file address starting from the current folder	3
Repository (repo)	A folder whose history Git is tracking	12
Script	A .py file run from start to finish	6

TERM	PLAIN MEANING	CHAPTER
Standard library	Packages that come built into a language	7
Syntax	The spelling and grammar rules of a language	6
Terminal / shell	Typing commands to the OS	4
Traceback	Python's error report, read bottom-up	10
Variable	A named box holding a value	6
Virtual environment	A separate set of packages for one project	8
Working directory	The folder a program or terminal is currently "in"	3

Further reading, in order

These go deeper than this book. The order follows the chapters above. All of these are free.

1. **Alongside Chapters 1–2:** [Crash Course Computer Science](#), 40 short animated episodes from transistors to the internet.
2. **After Chapter 4:** [Software Carpentry: The Unix Shell](#), terminal lessons written for researchers.
3. **After Chapter 6:** [Software Carpentry: Programming with Python](#), Python taught with real scientific data.
4. **After Chapter 8:** [The Missing Semester of Your CS Education](#) (MIT), the tools most courses skip.
5. **Chapter 11 onwards:** [Earth Lab: Earth Data Science and Project Pythia](#), Python for Earth scientists.

Books to grow into

These take you further once the basics feel comfortable. Most are free to read online.

How computers work

- *Code: The Hidden Language of Computer Hardware and Software*, Charles Petzold (2nd ed., 2022). Builds a computer from switches upward. Its free companion site, CodeHiddenLanguage.com, animates the circuits.
- *How Computers Really Work*, Matthew Justice. A hands-on tour of hardware and software, with small projects.
- *Operating Systems: Three Easy Pieces*, Remzi and Andrea Arpaci-Dusseau. Free. A clear, friendly OS textbook that also fits the MCA operating systems course.
- *Computer Networking: A Top-Down Approach*, Kurose and Ross (8th ed., Global Edition, ISBN 978-1-292-40546-9, the edition sold in India). The standard networking text, useful for the MCA networks course. The authors' [companion site](#) has free lecture videos and slides.

Programming

- *Think Python*, Allen Downey (3rd ed.). Free. Defines every term as it appears; ideal after this book.
- *Automate the Boring Stuff with Python*, Al Sweigart. Free. Practical scripts for files, spreadsheets and PDFs.

- [*Pro Git*](#), Scott Chacon and Ben Straub. Free. Version control, the next tool to learn after this book.

Data and geoscience

- [*Python for Data Analysis*](#), Wes McKinney (3rd ed.). Free online. Written by the creator of pandas.
- [*Python Data Science Handbook*](#), Jake VanderPlas. Free. NumPy, pandas, matplotlib and machine learning basics.
- [*Earth and Environmental Data Science*](#), Ryan Abernathey. Free. A university course on Python for Earth scientists.
- [*Geographic Data Science with Python*](#), Rey, Arribas-Bel and Wolf. Free. Spatial data, maps and spatial statistics.

A last piece of advice: when a resource stops making sense, come back to the matching chapter here, re-read it, and then continue. The missing piece is usually context, and context is what this book is for.